

UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA



Curso:

Cloud Computing (CS2032)

Proyecto: CloudCommerce - Plataforma de comercio electrónico basada en microservicios y arquitectura cloud

Integrantes Grupo:

Daniel Guillermo Sandoval Toro (202310533)

Jose Ernesto Guerrero Cueva (202210566)

Ian Jesus Rodríguez Santayana (202310184)

Vannia Fatima Curay Mauricio (201910133)

Mia Wood De la fuente chavez (202410085)

Profesor:

Mejia Fernandez, Oscar Rodolfo

ÍNDICE

1. INTRODUCCIÓN	3
1.1 Objetivos	3
1.2 Correspondencia con la rúbrica	3
2. ARQUITECTURA DE SOLUCIÓN	4
2.1 Componentes y servicios AWS	6
2.2 Recorrido de una petición.....	6
2.3 Decisiones de diseño y seguridad	7
3. BACKEND: MICROSERVICIOS	7
3.1 Resumen.....	7
3.2 ms-productos	8
3.3 ms-usuarios.....	8
3.4 ms-pedidos	9
3.5 ms-checkout	10
3.6 ms-analitica	11
3.7 Modelo de datos	12
3.8 Pruebas y documentación de las API.....	14
4. FRONTEND.....	15
5. DATA SCIENCE	19
5.1 Ingesta hacia S3.....	19
5.2 Catálogo de datos (AWS Glue)	20
5.3 Consultas y vistas en Athena	22
6. DESPLIEGUE EN AWS	26
7. LIMITACIONES Y TRABAJO FUTURO	29
8. CONCLUSIONES	30

1. INTRODUCCIÓN

CloudCommerce es un e-commerce simple construido como proyecto parcial del curso. Su propósito es demostrar el diseño, despliegue y análisis de una solución en la nube: un backend en 5 microservicios con tecnologías y bases de datos distintas, un frontend web que los consume, y un pipeline de Data Science que lleva los datos transaccionales a un data lake en S3 para consultarlos con Athena.

1.1 Objetivos

- Implementar microservicios REST independientes (Python, Java y Node.js) con bases de datos propias: MySQL, PostgreSQL y MongoDB.
- Desplegar el backend con redundancia: dos MV de producción con los cinco microservicios cada una, detrás de un balanceador de carga interno y de API Gateway, con las bases de datos en una MV privada.
- Publicar una aplicación web en AWS Amplify que consuma los cinco microservicios.
- Ingerir el 100 % de los datos hacia S3, catalogarlos con AWS Glue y analizarlos con consultas y vistas en Athena.

1.2 Correspondencia con la rúbrica

Ítem	Pts.	Dónde se documenta
Backend: microservicios	7	Sección 3 (más sección 6, despliegue)
Frontend: web	3	Sección 4
Data Science	5	Sección 5
Diagrama de Arquitectura de Solución	1	Sección 2, Figura 1
Exposición presencial	1	Presentación (archivo aparte)
Exposición virtual con ACL	3	Hito 1 (fuera de este informe)

El código completo y las guías de despliegue están en el repositorio público de GitHub:

<https://github.com/DaniSandt1/CloudCommerce> (README.md, DEPLOYMENT.md y el README de cada carpeta).

2. ARQUITECTURA DE SOLUCIÓN

La Figura 1 muestra la arquitectura final desplegada, con todos los servicios de AWS usados en Backend, Frontend y Data Science. Se lee de izquierda a derecha siguiendo una petición: el usuario entra por Amplify, API Gateway es la única entrada pública al backend y el ALB interno reparte el tráfico entre dos réplicas idénticas de los cinco microservicios. La franja de la derecha es el flujo de datos para analítica (S3, Glue y Athena).

CloudCommerce — Diagrama de Arquitectura de Solución (AWS)

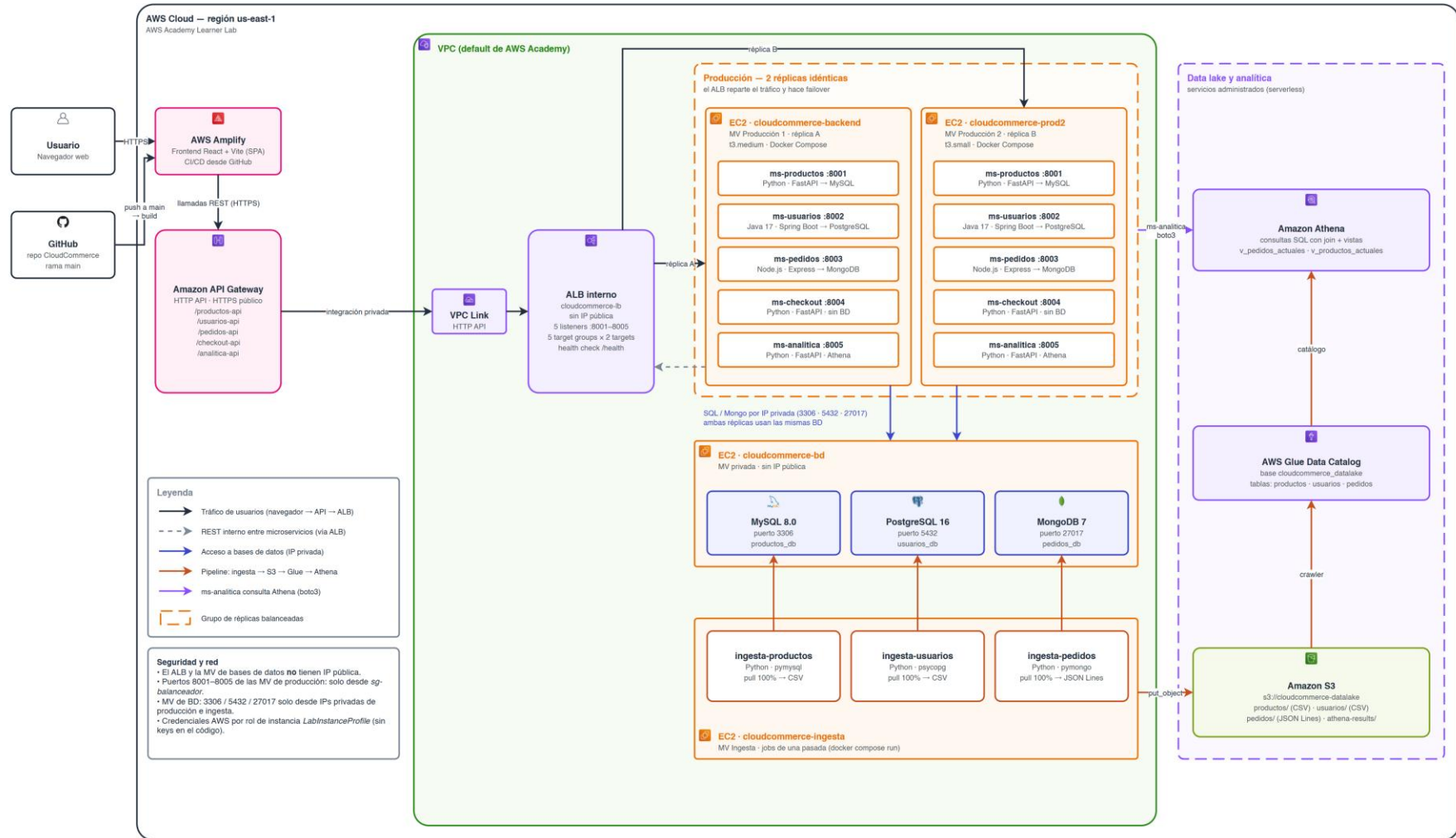


Figura 1. Diagrama de Arquitectura de Solución de CloudCommerce (arquitectura final con redundancia).

2.1 Componentes y servicios AWS

Capa	Servicio / recurso	Función
Frontend	AWS Amplify	Hospeda la SPA React + Vite. Monorepo (appRoot: frontend), build definido en amplify.yml ; las URLs de la API se inyectan con variables VITE_*_API_URL .
Entrada	Amazon API Gateway (HTTP API)	Punto público HTTPS con CORS. Una ruta por microservicio: /productos-api , /usuarios-api , /pedidos-api , /checkout-api y /analitica-api ; cada ruta reenvía el resto del path al microservicio.
Red privada	VPC Link cloudcommerce-vpclink + ALB interno cloudcommerce-lb	API Gateway llega al balanceador por integración privada. El ALB no tiene IP pública y expone un listener por microservicio (8001–8005). Cada target group tiene dos destinos, uno en cada MV de producción, con health check /health .
Cómputo	4 × Amazon EC2 (Ubuntu Server, Docker)	cloudcommerce-backend y cloudcommerce-prod2 (producción: dos réplicas idénticas con los 5 microservicios), cloudcommerce-bd (bases de datos) y cloudcommerce-ingesta (jobs de ingesta).
Datos transaccionales	MySQL 8, PostgreSQL 16, MongoDB 7 (contenedores en cloudcommerce-bd)	Una base por microservicio, en una MV privada sin IP pública, compartida por las dos réplicas.
Data lake	Amazon S3 cloudcommerce-datalake	Destino de la ingesta (prefijos productos/ , usuarios/ , pedidos/) y de los resultados de Athena.
Catálogo y consultas	AWS Glue Data Catalog + Amazon Athena	Base cloudcommerce_datalake con tablas, consultas SQL y vistas sobre los datos en S3.
Identidad	Rol IAM LabInstanceProfile	Credenciales de instancia para S3, Glue y Athena; no hay llaves en el código.

2.2 Recorrido de una petición

1. El usuario abre la aplicación servida por **Amplify**.
2. El navegador llama por HTTPS a **API Gateway** (por ejemplo, **/pedidos-api/...**).
3. La ruta usa una integración privada a través del **VPC Link** y llega al **ALB interno**.
4. El ALB reenvía según el puerto del listener al target group del microservicio, que entrega la petición a una de sus dos réplicas sanas (**cloudcommerce-backend** o **cloudcommerce-prod2**).
5. El microservicio consulta su base de datos por IP privada en **cloudcommerce-bd**.
6. **ms-pedidos** y **ms-checkout** consumen a los otros microservicios por HTTP a través del DNS interno del ALB (no de una IP fija), así que el failover también cubre las llamadas internas.

2.3 Decisiones de diseño y seguridad

- **Bases de datos aisladas:** viven en una tercera MV sin IP pública; sus security groups sólo admiten las IP privadas de las MV de producción y de ingesta.
- **Redundancia real:** cada MV de producción ejecuta los cinco microservicios. Si una réplica falla, el health check la marca como Unhealthy y el ALB envía todo el tráfico a la otra.
- **Entrada única:** los puertos 8001–8005 de las MV de producción aceptan tráfico solo desde el security group del balanceador.
- **Ingesta desacoplada:** corre en su propia MV y lee las bases por IP privada, sin afectar la producción.
- **Datos sensibles:** la ingesta de usuarios no exporta password_hash.
- **Analítica desacoplada de AWS:** ms-analitica usa un cliente de Athena abstraído: modo mock para desarrollo local y pruebas, y modo real (ATHENA_MODE=real) en producción.

3. BACKEND: MICROSERVICIOS

El dominio es un e-commerce. Cada microservicio se ejecuta en un contenedor Docker, expone documentación Swagger/OpenAPI y responde /health para los health checks del balanceador.

Código fuente de los 5 microservicios:

<https://github.com/DaniSandt1/CloudCommerce/tree/main/backend>

3.1 Resumen

Microservicio	Tecnología	Base de datos	Puerto	Rol
ms-productos	Python 3.12 · FastAPI · SQLAlchemy	MySQL 8	8001	Catálogo de productos y categorías
ms-usuarios	Java 17 · Spring Boot 3.3	PostgreSQL 16	8002	Clientes y sus direcciones
ms-pedidos	Node.js 22 · Express 5 · Mongoose	MongoDB 7	8003	Órdenes de compra
ms-checkout	Python · FastAPI · httpx	Sin BD	8004	Orquesta productos + usuarios + pedidos
ms-analitica	Python · FastAPI · boto3	Athena (sin BD transaccional)	8005	Consultas analíticas sobre el data lake

Los tres servicios con base de datos se cargaron con 20 000 registros ficticios cada uno (Faker en productos, DataSeeder en usuarios y seedPedidos.js en pedidos). ms-pedidos consume a ms-productos

y ms-usuarios, y ms-checkout consume a los otros tres, lo que cumple el requisito de comunicación entre microservicios.

3.2 ms-productos

Método	Ruta	Descripción
GET	/health	Estado y total de productos
GET / POST	/categorias	Lista y crea categorías
GET	/productos	Lista paginada (page, size) con filtros categoria_id y q (búsqueda por nombre)
GET	/productos/{id}	Detalle de un producto
POST	/productos	Crea un producto
PUT	/productos/{id}	Actualiza un producto
DELETE	/productos/{id}	Elimina un producto (204)

3.3 ms-usuarios

Método	Ruta	Descripción
GET	/health	Estado y total de usuarios
GET	/usuarios	Lista paginada con búsqueda q por nombre o email
GET / PUT / DELETE	/usuarios/{id}	Consulta, actualiza o elimina un usuario
POST	/usuarios	Crea un usuario (201). password_hash es solo de escritura: nunca se devuelve
GET / POST	/usuarios/{id}/direcciones	Lista y agrega direcciones del usuario

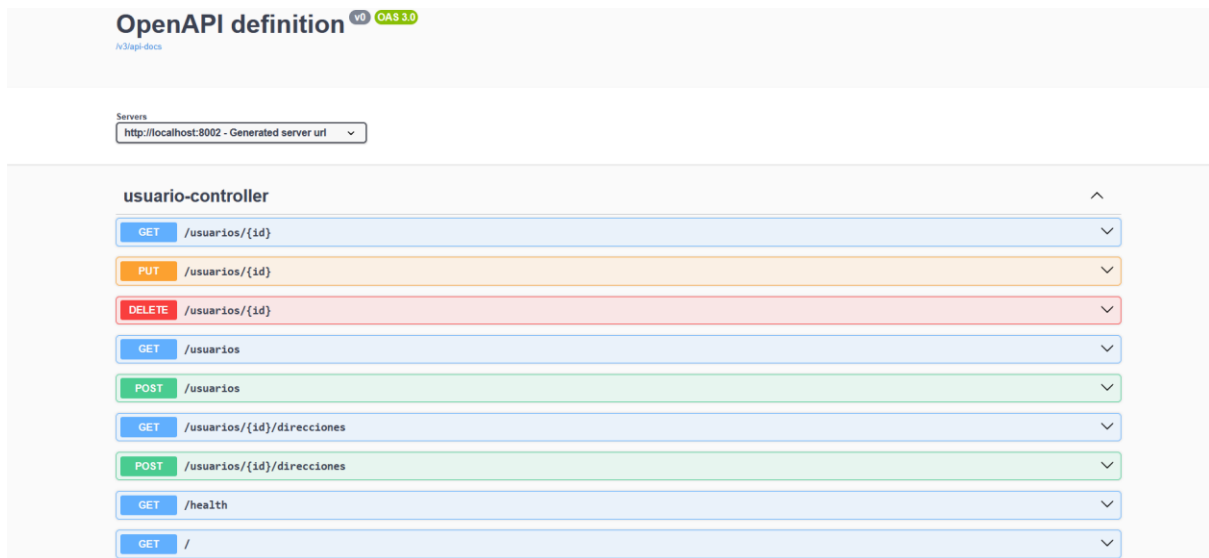


Figura 2. Swagger UI de ms-usuarios.

3.4 ms-pedidos

Método	Ruta	Descripción
GET	/health	Estado del servicio
POST	/pedidos	Crea un pedido: valida que exista el usuario (ms-usuarios), toma nombre y precio de cada producto (ms-productos) y calcula el total. Estado inicial pendiente
GET	/pedidos/{id}	Detalle de un pedido por su ID de MongoDB
GET	/usuarios/{id_usuario}/pedidos	Pedidos de un usuario

El precio unitario queda congelado dentro de cada ítem, de modo que el historial no cambia si el producto se modifica luego. Estados posibles: pendiente, pagado, enviado y cancelado.

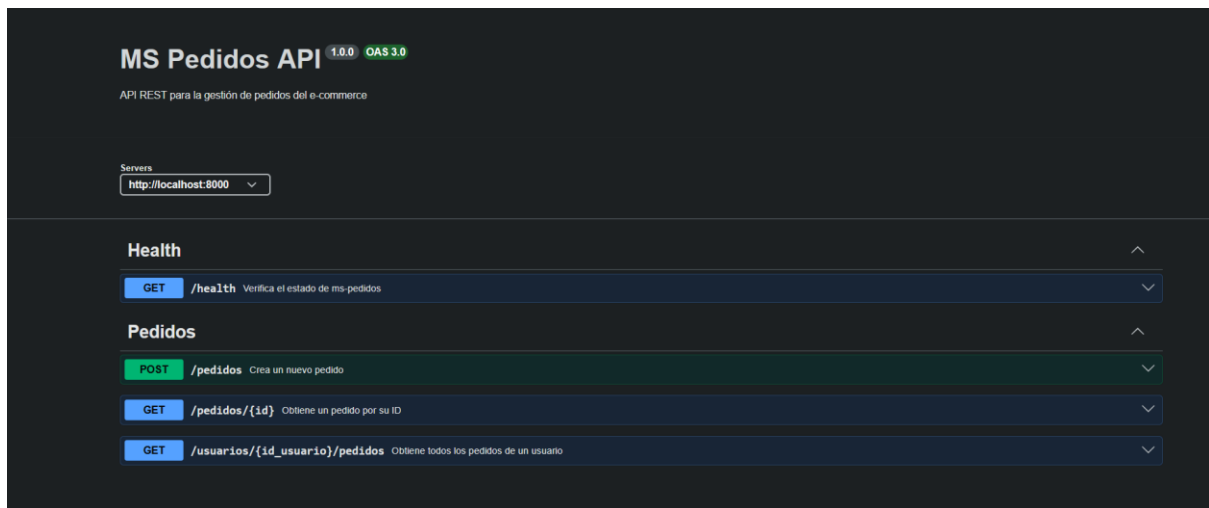


Figura 3. Swagger UI de ms-pedidos.

3.5 ms-checkout

Método	Ruta	Descripción
GET	/health	Estado del servicio (database: none)
POST	/checkout/resumen	Valida usuario, productos y stock y calcula el total, sin crear pedido
POST	/checkout/confirmar	Repite la validación y crea el pedido en ms-pedidos (201)
GET	/checkout/pedidos/{id_pedido}	Pedido enriquecido con datos del usuario y nombres de producto

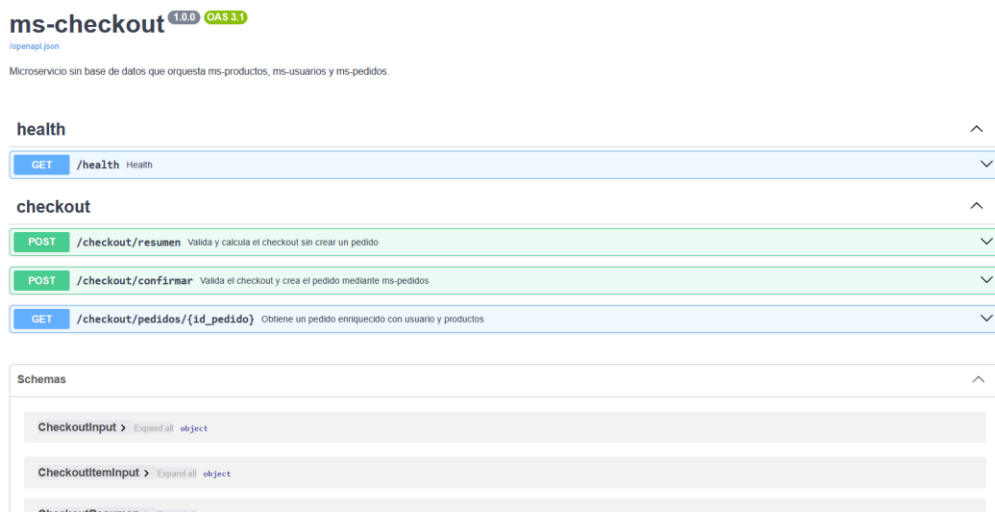


Figura 4. Swagger UI de ms-checkout.

3.6 ms-analitica

Método	Ruta	Descripción
GET	/health	Estado del servicio
GET	/analitica/ventas-por-categoria	Monto vendido por categoría de producto
GET	/analitica/top-clientes?limit=10	Cientes con mayor monto comprado (limit entre 1 y 100)
GET	/analitica/pedidos-por-rango?fecha_inicio=&fecha_fin=	Cantidad y monto de pedidos en un rango de fechas (formato YYYY-MM-DD , inclusivo)

En producción, ms-analitica corre con `ATHENA_MODE=real`: con boto3 ejecuta las consultas sobre la base `cloudcommerce_datalake`, tomando el último snapshot de cada tabla (igual que las vistas de la sección 5.3), en el workgroup `primary` y con resultados en `s3://cloudcommerce-datalake/athena-results/`. Las credenciales vienen del rol `LabInstanceProfile` de la instancia. En desarrollo local se usa `ATHENA_MODE=mock`, que responde con datos de ejemplo sin conectarse a AWS.

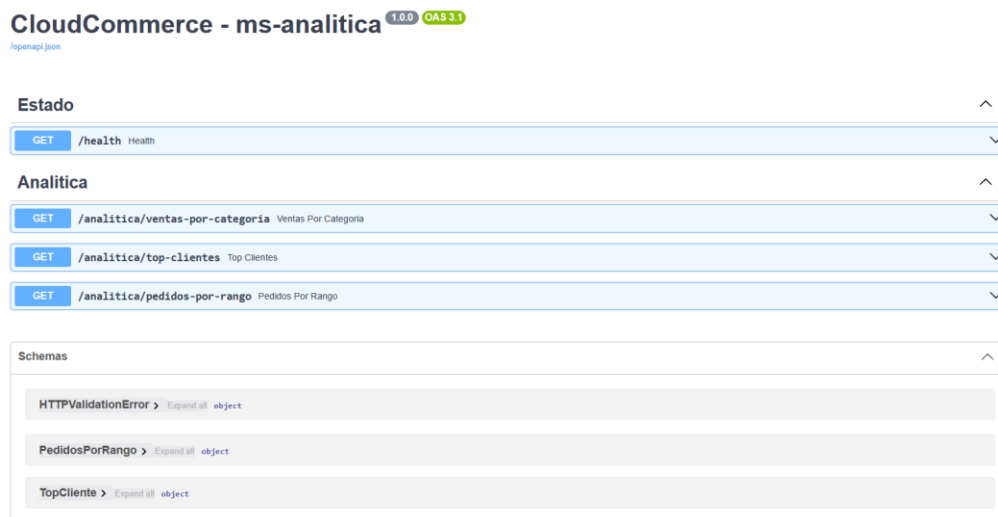


Figura 5. Swagger UI de ms-analitica.

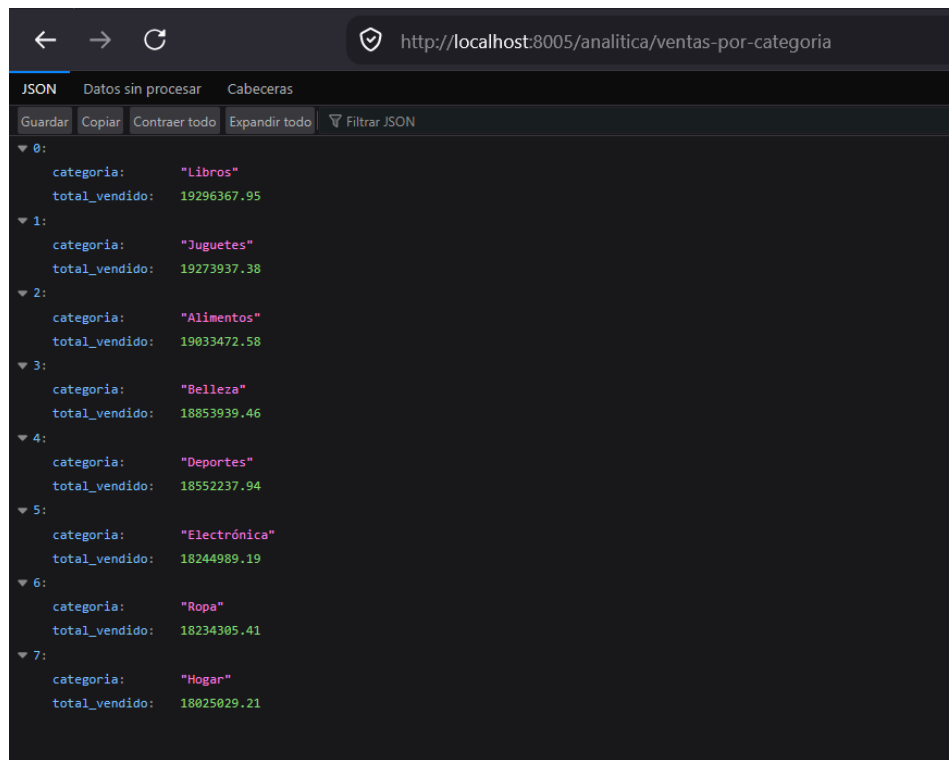


Figura 6. Respuesta de GET /analitica/ventas-por-categoria.

3.7 Modelo de datos

La Figura 7 muestra el modelo de datos de los tres microservicios con base propia. En PostgreSQL, un usuario tiene cero o más direcciones; en MySQL, cada producto pertenece a una categoría. La base de ms-pedidos es documental (MongoDB): los items van embebidos en cada pedido y el precio unitario queda congelado. Las referencias a usuario y producto cruzan bases distintas, por eso no son claves foráneas: ms-pedidos las valida llamando a ms-usuarios y ms-productos antes de guardar.

CloudCommerce — Diagrama E/R de las bases de datos de los microservicios

Las 3 bases viven en la MV privada **cloudcommerce-bd**. Cada microservicio es dueño de su base (no comparten tablas).

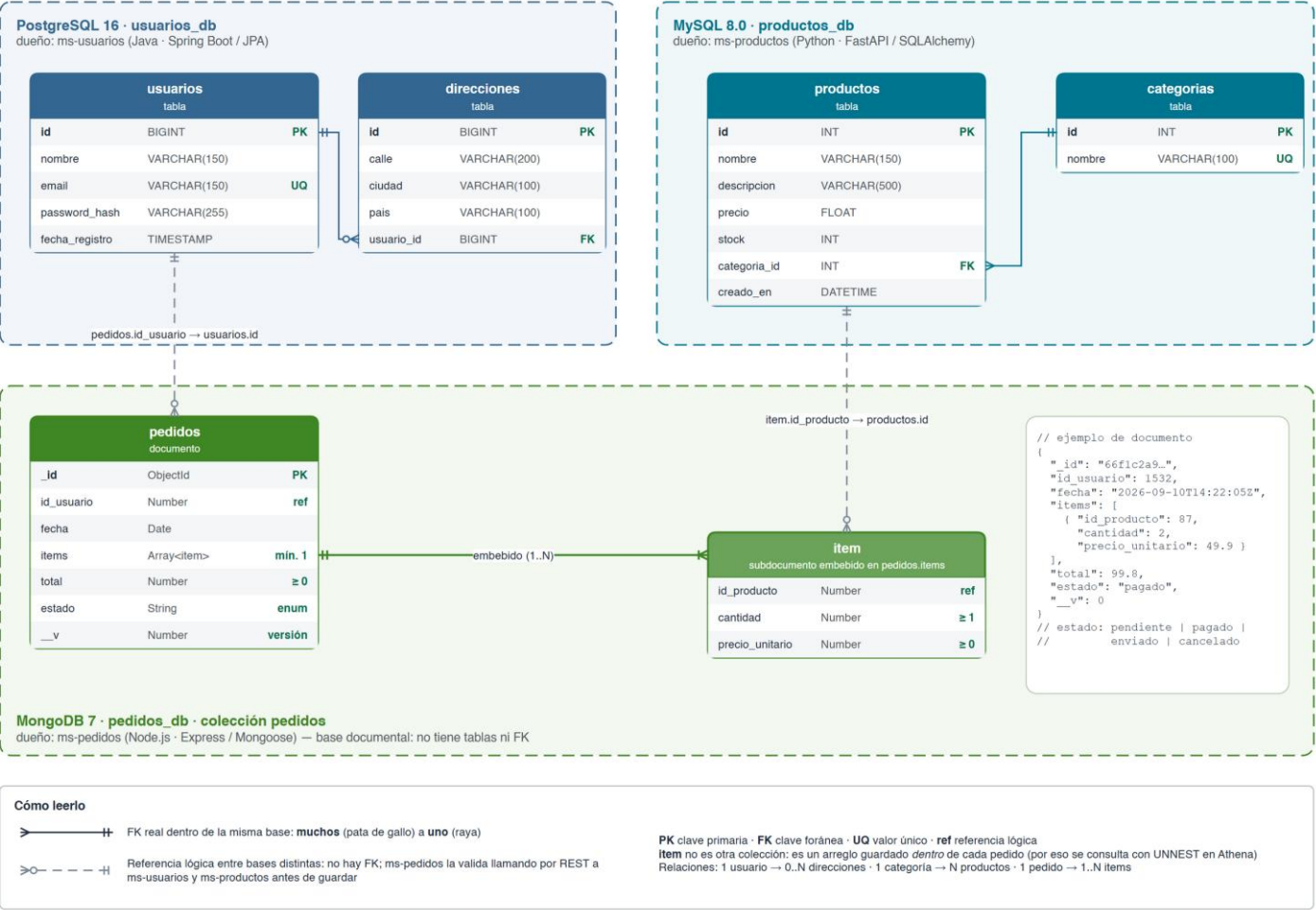


Figura 7. Diagrama E/R de las bases de los microservicios: PostgreSQL (ms-usuarios), MySQL (ms-productos) y colección pedidos de MongoDB (ms-pedidos).

Estructura JSON de la colección ‘pedidos’ (MongoDB)

```
{
  "_id": "ObjectId",
  "id_usuario": 123,
  "fecha": "2026-09-10T10:00:00Z",
  "items": [
    { "id_producto": 45, "cantidad": 2, "precio_unitario": 19.90 }
  ],
  "total": 39.80,
  "estado": "pendiente",
  "__v": 0
}
```

El campo __v lo agrega Mongoose para versionar el documento; por eso aparece también como columna en la tabla pedidos del catálogo de Glue.

3.8 Pruebas y documentación de las API

- Pruebas unitarias en ms-checkout y ms-analitica (python -m unittest), sin depender de AWS ni de otros servicios.
- Swagger UI en cada microservicio y colecciones de Postman en ms-productos y ms-analitica.
- Carga de datos verificable por /health (total_productos, total_usuarios) y scripts/countPedidos.js.

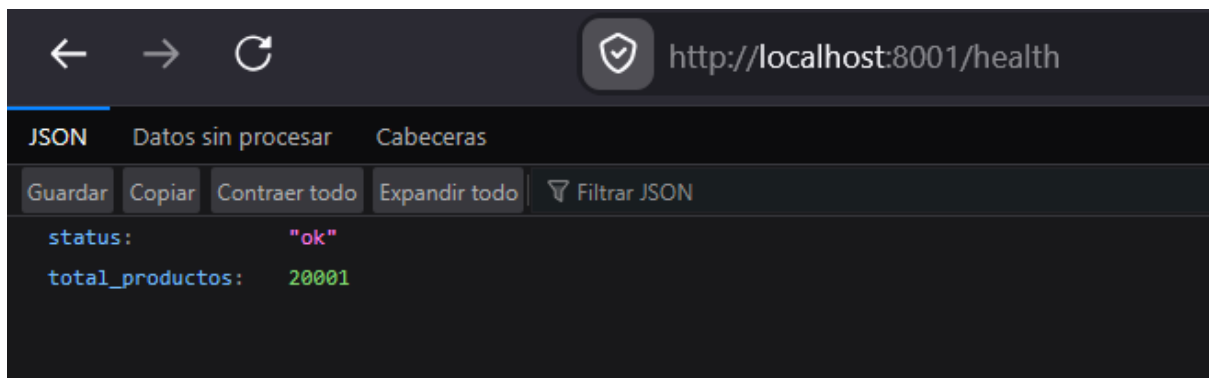


Figura 8. GET /health de ms-productos con el total de productos cargados.

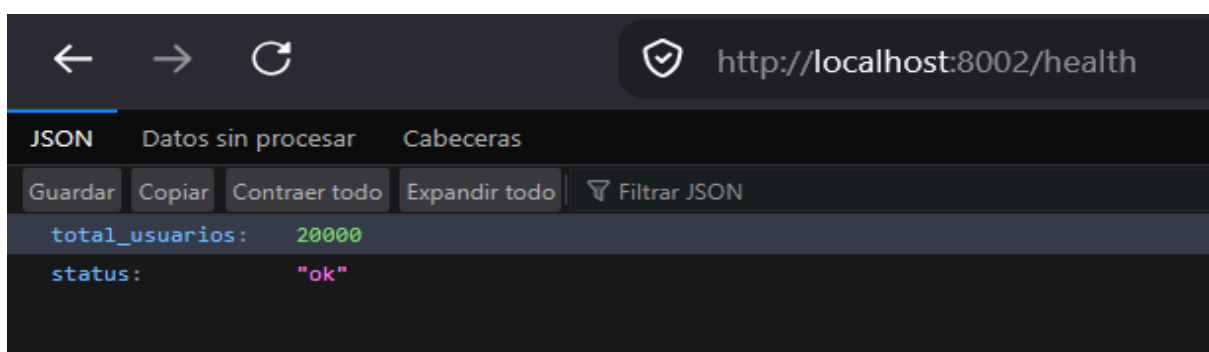


Figura 9. GET /health de ms-usuarios con los 20 000 usuarios cargados.

```
Last login: Tue Sep 22 00:49:50 2026 from 45.236.45.48
ubuntu@ip-172-31-74-245:~$ cd ~/CloudCommerce/backend/docker-compose
ubuntu@ip-172-31-74-245:~/CloudCommerce/backend/docker-compose$ docker compose exec ms-pedidos node scripts/countPedidos.js
Cantidad de pedidos: 20000

Pedido más reciente:
{
  "id": "6aadd2c3649326e2cc0bb9df",
  "id_usuario": 16403,
  "fecha": "2026-09-19T00:09:39.222Z",
  "items": [
    {
      "id_producto": 14567,
      "cantidad": 4,
      "precio_unitario": 1507.27
    },
    {
      "id_producto": 834,
      "cantidad": 2,
      "precio_unitario": 207.1
    }
  ],
  "total": 6443.28,
  "estado": "pagado",
  "v": 0
}
```

i-02e179efe1b736c20 (cloudcommerce-prod2)

Figura 10. Conteo de pedidos con scripts/countPedidos.js en cloudcommerce-prod2 (20 000 pedidos).

4. FRONTEND

Código fuente del frontend: <https://github.com/DaniSandt1/CloudCommerce/tree/main/frontend>

La aplicación web usa React 18, Vite 5 y Tailwind CSS 3, con íconos de lucide-react, y está desplegada en AWS Amplify. Consume los cinco microservicios a través de API Gateway; las URLs base se configuran con variables de entorno (VITE_PRODUCTOS_API_URL, VITE_USUARIOS_API_URL, VITE_PEDIDOS_API_URL, VITE_CHECKOUT_API_URL y VITE_ANALITICA_API_URL).

Pestaña	Funcionalidad	Endpoints que invoca
Productos	Catálogo paginado con búsqueda por nombre, filtro por categoría, detalle en modal y alta de producto	GET /productos, GET /categorias, POST /productos
Usuarios	Listado paginado con búsqueda por nombre/email y alta de usuario	GET /usuarios, POST /usuarios
Pedidos	Búsqueda de pedidos por ID de usuario, detalle y flujo “Nuevo pedido”: primero resumen (precios y stock) y luego confirmación	GET /usuarios/{id}/pedidos, GET /pedidos/{id}, POST /checkout/resumen, POST /checkout/confirmar

Analítica	Ventas por categoría, top de clientes (5, 10, 20 o 50) y pedidos por rango de fechas	GET /analitica/*
-----------	--	------------------

Despliegue en Amplify

El repositorio de GitHub está conectado a Amplify como monorepo (appRoot: frontend). Cada push a main dispara el build (npm ci y npm run build) y publica el contenido de dist/.

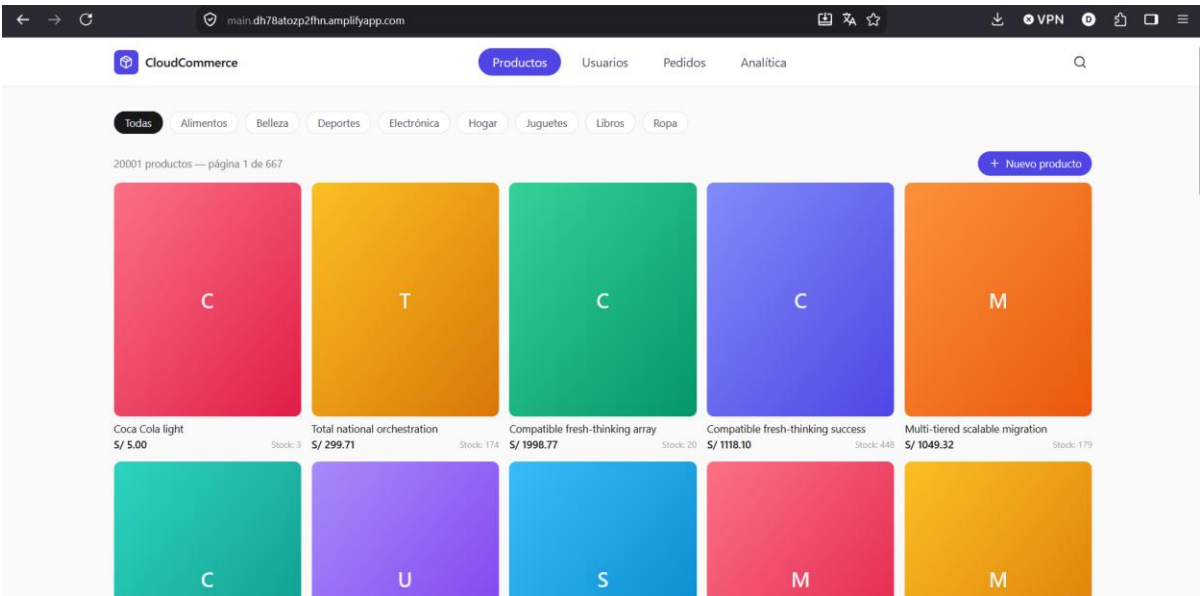


Figura 11. Pestaña Productos del frontend.

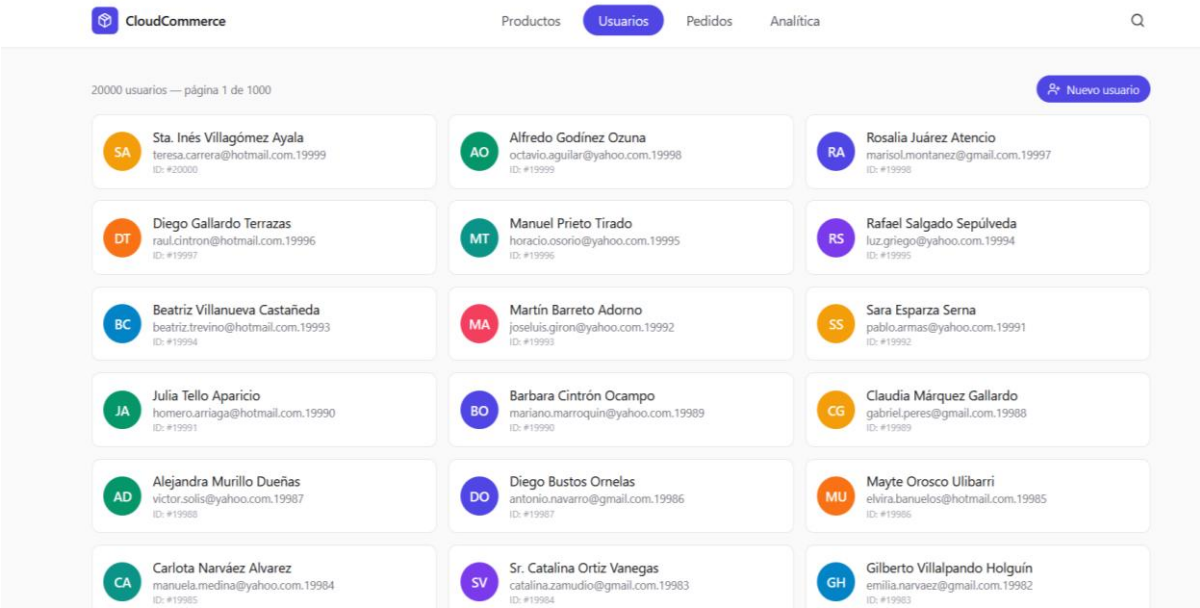


Figura 12. Pestaña Usuarios del frontend.

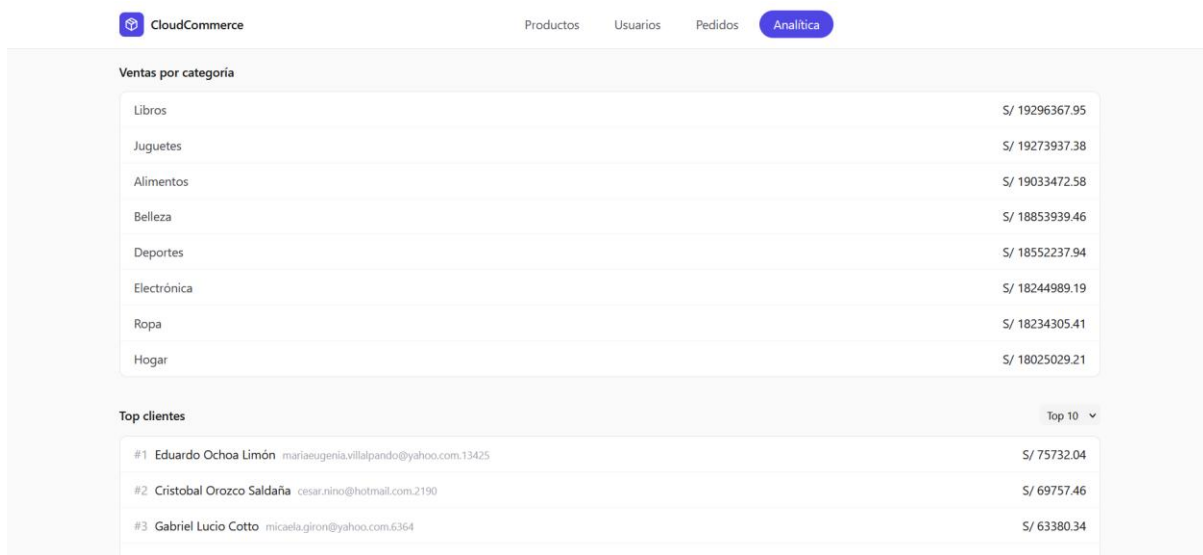


Figura 13. Pestaña Analítica del frontend.

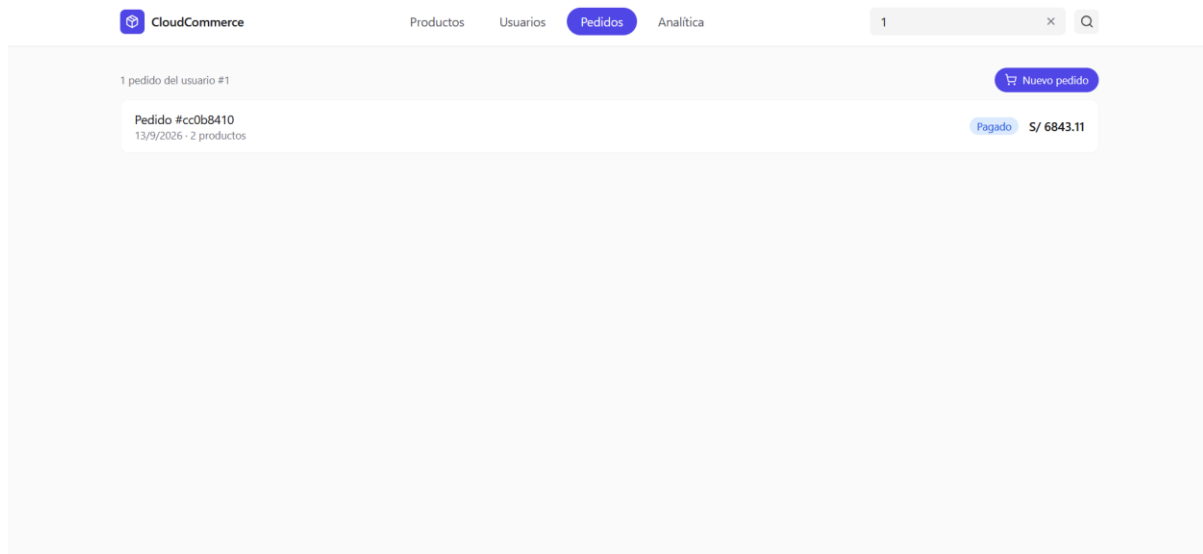


Figura 14. Pestaña Pedidos del frontend.

Variables de entorno

Las variables de entorno son pares de claves o valores que contienen cualquier valor constante que la aplicación necesite en el momento de la compilación, por ejemplo, los detalles de la conexión a la base de datos o las claves de la API de terceros.

Más información

Ramificación	Variable	Valor
Todas las ramificaciones	AMPLIFY_DIFF_DEPLOY	false
Todas las ramificaciones	AMPLIFY_MONOREPO_API_ROOT	frontend
Todas las ramificaciones	VITE_ANALITICA_API_URL	https://b1rvjwotg.execute-api.us-east-1.amazonaws.com/analitica-api
Todas las ramificaciones	VITE_CHECKOUT_API_URL	https://b1rvjwotg.execute-api.us-east-1.amazonaws.com/checkout-api
Todas las ramificaciones	VITE_PEDIDOS_API_URL	https://b1rvjwotg.execute-api.us-east-1.amazonaws.com/pedidos-api
Todas las ramificaciones	VITE_PRODUCTOS_API_URL	https://b1rvjwotg.execute-api.us-east-1.amazonaws.com/productos-api
Todas las ramificaciones	VITE_USUARIOS_API_URL	https://b1rvjwotg.execute-api.us-east-1.amazonaws.com/usuarios-api

Rows per page: 15

Figura 15. Variables de entorno VITE_*_API_URL en AWS Amplify.

CloudCommerce

ID de la aplicación: dh78atozp2fhn

Pasarse a producción (0 de 3 pasos completados)

- 1 Agregar un dominio personalizado**
Utilice su propio dominio personalizado con HTTPS gratuito para proporcionar una URL segura y accesible para su aplicación.
[Agregar dominio personalizado](#)
- 2 Activar protecciones de firewall**
AWS Web Application Firewall (WAF) ofrece restricciones de tráfico web para Amplify Hosting.
[Activar firewall](#)
- 3 Conectar nuevas ramificaciones**
Conectar otra ramificación de su repositorio de Git para configurar varios entornos.
[Conectar una nueva ramificación](#)

Ramificaciones 1 [+ Agregar ramificación](#)

Ramificación	Implementación	Última implementación	Última confirmación
main	Implementación realizada	Hace 22 horas	Ajustar ingesta de pedidos a J... / CloudCommerce:main

★ Ramificación de producción

Figura 16. Aplicación CloudCommerce en AWS Amplify con la rama main desplegada.

5. DATA SCIENCE

Código fuente de la ingesta y el catálogo:

<https://github.com/DaniSandt1/CloudCommerce/tree/main/data-science>

5.1 Ingesta hacia S3

Tres contenedores Docker en Python corren en la MV cloudcommerce-ingesta, separada de producción. Cada uno hace un pull del 100 % de su fuente y sube un archivo con marca de tiempo al bucket cloudcommerce-datalake. Son jobs de una sola pasada (docker compose run --rm <servicio>) y usan el rol de la instancia para acceder a S3.

Contenedor	Fuente	Formato	Destino en S3
ingesta-productos	MySQL (productos + categorías)	CSV	productos/productos_<timestamp>.csv
ingesta-usuarios	PostgreSQL (usuarios + direcciones)	CSV, una fila por dirección; password_hash	usuarios/usuarios_<timestamp>.csv
ingesta-pedidos	MongoDB (pedidos)	JSON Lines, un pedido por línea con items embebidos	pedidos/pedidos_<timestamp>.jsonl

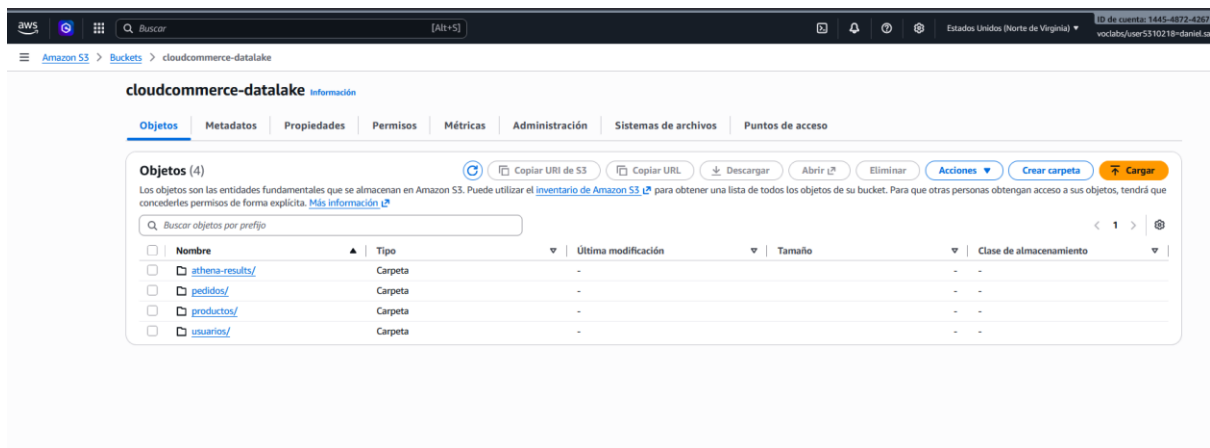


Figura 17. Bucket cloudcommerce-datalake con las carpetas de la ingesta y los resultados de Athena.

5.2 Catálogo de datos (AWS Glue)

Un crawler de AWS Glue recorrió las tres carpetas de S3 y creó la base cloudcommerce_datalake con una tabla por carpeta: productos, usuarios y pedidos (Figura 18). Tres tablas bastan para todas las consultas: las categorías llegan desnormalizadas en productos (categoria_nombre), las direcciones llegan aplanadas en usuarios y los items van anidados dentro de pedidos. El campo items se modela como `array<struct<id_producto:bigint,cantidad:int,precio_unitario:double>>` para poder expandirlo con UNNEST en Athena.

Dos detalles del esquema condicionan las consultas. Primero, usuarios tiene una fila por dirección, así que su id puede repetirse y se deduplica antes de unir. Segundo, los tipos de las claves no coinciden entre tablas (id_usuario es int en pedidos, mientras que id es bigint en usuarios y productos), por lo que los JOIN convierten ambos lados con `CAST(... AS BIGINT)`.

CloudCommerce — Diagrama E/R del catálogo de datos (AWS Glue)

Base `cloudcommerce_datalake` · 3 tablas creadas por el crawler sobre `s3://cloudcommerce-datalake` · consultadas con Athena

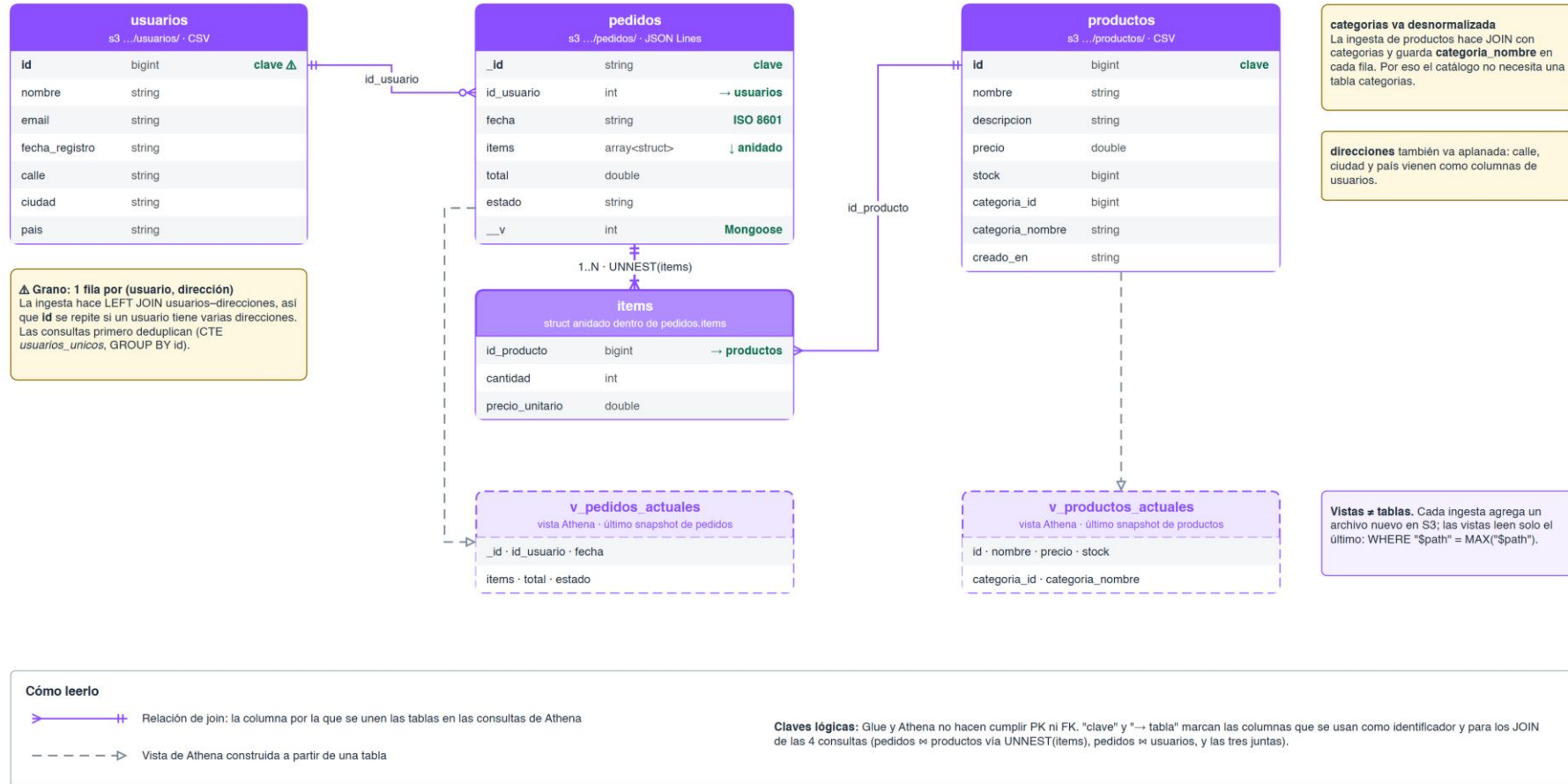


Figura 18. Diagrama E/R del catálogo de datos en AWS Glue (base `cloudcommerce_datalake`).

Column Name	Data Type
_id	string
id_usuario	int
fecha	string
items	string
total	double
estado	string
_id	int

Column Name	Data Type
_id	string
nombre	string
descripcion	string
precio	double
stock	integer
categoria_id	integer
categoria_nombre	string
stock_id	string

Column Name	Data Type
_id	string
nombre	string
email	string
fecha_registro	string
telefono	string
password	string
password_confirmado	string
password_confirmado	string

Figura 19. Esquemas de las tablas pedidos, productos y usuarios en AWS Glue.

Property	Value
Name	cloudcommerce_datalake
Description	-
Location	-
Created on (UTC)	September 20, 2026 at 23:25:01

Name	Database	Location	Classification	Deprecated	View data	Data quality	Column statistics
pedidos	cloudcommerce_datalake	s3://cloudcommerce-datalak	JSON	-	Table data	View data quality	View statistics
productos	cloudcommerce_datalake	s3://cloudcommerce-datalak	CSV	-	Table data	View data quality	View statistics
usuarios	cloudcommerce_datalake	s3://cloudcommerce-datalak	CSV	-	Table data	View data quality	View statistics
v_pedidos_actuales	cloudcommerce_datalake	-	-	-	-	View data quality	View statistics
v_productos_actuales	cloudcommerce_datalake	-	-	-	-	View data quality	View statistics

Figura 20. Base cloudcommerce_datalake en AWS Glue con sus tres tablas.

5.3 Consultas y vistas en Athena

Como cada corrida de ingesta agrega un archivo nuevo, las tablas acumulan varias versiones del mismo dato. Por eso las dos vistas seleccionan sólo el último snapshot (MAX("\$path")) y las consultas parten de ellas. El CSV de usuarios trae una fila por dirección, así que se deduplica por id antes de unir.

Vista 1 — v_pedidos_actuales

```
CREATE OR REPLACE VIEW v_pedidos_actuales AS
SELECT _id, id_usuario, fecha, items, total, estado
FROM pedidos
WHERE "$path" = (SELECT MAX("$path") FROM pedidos);
```

Vista 2 — v_productos_actuales

```
CREATE OR REPLACE VIEW v_productos_actuales AS
SELECT id, nombre, precio, stock, categoria_id, categoria_nombre
FROM productos
WHERE "$path" = (SELECT MAX("$path") FROM productos);
```

Datos

Origen de datos

AwsDataCatalog

Catálogo

Ningún elemento

Base de datos

cloudcommerce_datalake

Tablas y vistas

Crear

⚙️

🔍

Filtrar tablas y vistas

▼ Tablas (3)

pedidos

productos

usuarios

▼ Vistas (2)

v_pedidos_actuales

v_productos_actuales

Consulta 1

Consulta 2

Consulta 3

Consulta 4

```

1 -- VISTA 1: v_pedidos_actuales
2 -- Último snapshot de pedidos (una fila por pedido, items anidados).
3 -- Cada corrida de ingesta sube un archivo nuevo con timestamp; esta vista
4 -- evita contar pedidos duplicados de corridas anteriores.
5 -----
6 CREATE OR REPLACE VIEW v_pedidos_actuales AS
7 SELECT _id, id_usuario, fecha, items, total, estado
8 FROM pedidos
9 WHERE "$path" = (SELECT MAX("$path") FROM pedidos);
10 -----
11 -- VISTA 2: v_productos_actuales
12 -- Último snapshot del catálogo de productos (incluye categoria_nombre).
13 -----
14 CREATE OR REPLACE VIEW v_productos_actuales AS
15 SELECT id, nombre, precio, stock, categoria_id, categoria_nombre

```

SQL Ln 17, Col 54

Ejecutar

Explicar

Cancelar

Borrar

Crear

Resultados de la consulta

Estado de la consulta

Completado

La consulta se ha realizado correctamente.

Figura 21. Creación de las vistas `v_pedidos_actuales` y `v_productos_actuales` en Athena.

Consulta 1 — Ventas por categoría (pedidos + productos)

```

SELECT p.categoria_nombre AS categoria,
       SUM(CAST(i.cantidad AS BIGINT)) AS unidades,
       ROUND(SUM(CAST(i.cantidad AS DOUBLE) * CAST(i.precio_unitario AS DOUBLE)), 2) AS
total_vendido
FROM v_pedidos_actuales o
CROSS JOIN UNNEST(o.items) AS t(i)
JOIN v_productos_actuales p
  ON CAST(p.id AS BIGINT) = CAST(i.id_producto AS BIGINT)
GROUP BY p.categoria_nombre
ORDER BY total_vendido DESC;

```

1	-- CONSULTA 1: ventas por categoría
2	-- Una pedidos (items desanidados) con productos.
3	
4	
5	SELECT p.categoria_nombre AS categoria,
6	SUM(CAST(i.cantidad AS BIGINT)) AS unidades,
7	ROUND(SUM(CAST(i.cantidad AS DOUBLE) * CAST(i.precio_unitario AS DOUBLE)), 2) AS total_vendido
8	FROM v_pedidos_actuales o
9	CROSS JOIN UNNEST(o.items) AS t(i)
10	JOIN v_productos_actuales p
11	ON CAST(p.id AS BIGINT) = CAST(i.id_producto AS BIGINT)
12	GROUP BY p.categoria_nombre
13	ORDER BY total_vendido DESC;

SQLLn 13, Col 29

Ejecutar de nuevo

Explicar i?

Cancelar

Borrar

Crear

Resultados de la consultaEstado de la consulta

Completado

Resultados (8)

QFilas de búsqueda

#	▼	categoria	▼	unidades	▼	total_vendido
1		Libros		19023		1.929636795E7
2		Juguetes		19202		1.927393738E7
3		Alimentos		19208		1.903347258E7
4		Belleza		18994		1.885393946E7
5		Deportes		18656		1.855223794E7
6		Electrónica		18678		1.824498919E7
7		Ropa		18213		1.823430541E7
8		Hogar		18116		1.802502921E7

Figura 22. Resultado de la Consulta 1 en Athena.

Consulta 2 — Top 10 clientes por monto (pedidos + usuarios)

```

WITH usuarios_unicos AS (
  SELECT id, MIN(nombre) AS nombre, MIN(email) AS email
  FROM usuarios
  WHERE "$path" = (SELECT MAX("$path") FROM usuarios)
  GROUP BY id
)
SELECT u.id AS id_usuario, u.nombre, u.email,
       COUNT(*) AS cantidad_pedidos,
       ROUND(SUM(CAST(o.total AS DOUBLE)), 2) AS total_comprado
FROM v_pedidos_actuales o
JOIN usuarios_unicos u
  ON CAST(u.id AS BIGINT) = CAST(o.id_usuario AS BIGINT)
GROUP BY u.id, u.nombre, u.email
ORDER BY total_comprado DESC
LIMIT 10;

```

Consulta 1

Consulta 2

Consulta 3

Consulta 4

```
2 -- CONSULTA 2: top 10 clientes por monto comprado
3 -- Una petición con usuarios: El CSV de usuarios trae una fila por
4 -- dirección, por eso se desduplica por la antes del 2020.
5
6 -----
7 SELECT id, MIN(nombre) AS nombre, MIN(email) AS email
8 FROM usuarios
9 WHERE "spath" = (SELECT MAX("spath") FROM usuarios)
10 GROUP BY id
11 )
12 SELECT u.id AS id_usuario, u.nombre, u.email,
13        COUNT(*) AS cantidad_pedidos,
14        ROUND(SUM(CAST(o.total AS DOUBLE)), 2) AS total_comprado
15 FROM v_pedidos_actuales o
16 JOIN usuarios_unicos u
SQL Ln 8, Col 8
```

Ejecutar de nuevo

Explicar 1

Cancelar

Borrar

Crear

Volver a utilizar los resultados de la consulta

hace 60 minutos

Resultados de la consulta

Estado de la consulta

Completado

Tiempo en cola: 86 ms

Tiempo de ejecución: 1.408 seg

Datos analizados: 20.23 MB

Resultados (10)

Copiar

Descargar resultados en formato CSV

Q: Filas de búsqueda

#	id_usuario	nombre	email	cantidad_pedidos	total_comprado
1	13426	Eduardo Ochoa Limón	mariaevgentia.villalpando@yahoo.com.13425	5	75732.04
2	2191	Cristobal Orzoco Saldaña	cesar.rino@hotmail.com.2190	4	69757.46
3	6365	Gabriel Lucio Cotto	micarla.giron@yahoo.com.6364	7	63380.34
4	17305	Lorena Oquendo Romero	pablo.borajin@yahoo.com.17304	4	63084.07
5	17461	Sra. Fernando Valle Robles	joaquin.sainz@gmail.com.17460	5	60280.77
6	10092	Sra. Eva Armenta Arevalo	soledad.carbajal@gmail.com.10091	6	59403.1
7	3513	Sra. Carla Villarreal Cardona	gilberto.murillo@gmail.com.3512	4	59142.38
8	19208	Natalia Nazario Niño	maricristina.gomez@yahoo.com.19207	4	59107.09
9	799	Agustín Sedillo Agosto	javier.lopez@yahoo.com.798	4	58357.96
10	16830	Marisol Rangel Sierra	francisco.gallardo@yahoo.com.16829	3	57849.29

Figura 23. Resultado de la Consulta 2 en Athena.

Consulta 3 — Top 10 productos más vendidos (pedidos + productos)

```

SELECT p.id AS id_producto, p.nombre, p.categoria_nombre AS categoria,
SUM(CAST(i.cantidad AS BIGINT)) AS unidades_vendidas,
ROUND(SUM(CAST(i.cantidad AS DOUBLE) * CAST(i.precio_unitario AS DOUBLE)), 2) AS
ingresos
FROM v_pedidos_actuales o
CROSS JOIN UNNEST(o.items) AS t(i)
JOIN v_productos_actuales p
ON CAST(p.id AS BIGINT) = CAST(i.id_producto AS BIGINT)
GROUP BY p.id, p.nombre, p.categoria_nombre
ORDER BY unidades_vendidas DESC
LIMIT 10;

```

1

-- CONSULTA 3: top 10 productos más vendidos

2

-- Una petición (items) con productos.

3

4

SELECT p.id AS id_producto, p.nombre, p.categoria_nombre AS categoria,

5

SUM(CAST(i.cantidad AS BIGINT)) AS unidades_vendidas,

6

ROUND(SUM(CAST(i.cantidad AS DOUBLE) * CAST(i.precio_unitario AS DOUBLE)), 2) AS ingresos

7

FROM v_pedidos_actuales o

8

CROSS JOIN UNNEST(o.items) AS t(i)

9

JOIN v_productos_actuales p

10

ON CAST(p.id AS BIGINT) = CAST(i.id_producto AS BIGINT)

11

GROUP BY p.id, p.nombre, p.categoria_nombre

12

ORDER BY unidades_vendidas DESC

13

LIMIT 10;

14

SQLLn 15, Col 1

Ejecutar de nuevo

Explicar 1?

Cancelar

Borrar

Crear

Volver a utilizar los resultados

Resultados de la consulta

Estado de la consulta

Completado

Tiempo en cola: 63 ms

Tiempo de ejecución: 1.943 seg

Datos analizados: 20.23 MB

Resultados (10)

Copiar

Descargar resultados

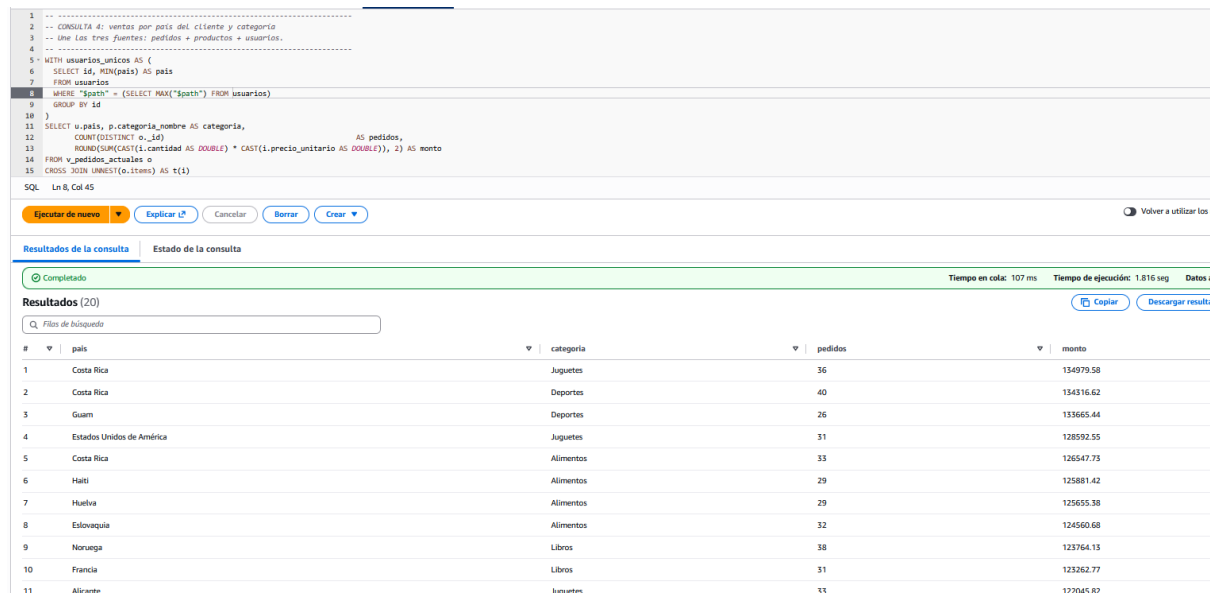
Q: Filas de búsqueda

#	id_producto	nombre	categoria	unidades_vendidas	ingresos
1	9101	Customizable multimedia utilization	Juguetes	39	39087.34
2	13421	Fully-configurable didactic encryption	Electrónica	37	196.1
3	11559	Front-line web-enabled help-desk	Libros	35	23929.62
4	2511	Visionary needs-based toolbar	Hogar	32	54912.64
5	9787	Centralized well-modulated task-force	Libros	32	45244.16
6	15749	Realigned directional infrastructure	Electrónica	32	31878.4
7	9934	Up-sized analyzing encoding	Libros	31	11875.79
8	17855	Profound global encoding	Hogar	31	60293.76
9	10595	Pre-emptive homogeneous core	Libros	31	14809.68
10	6335	Stand-alone maximized info-mediaries	Deportes	31	23997.72

Figura 24. Resultado de la Consulta 3 en Athena.

Consulta 4 — Ventas por país y categoría (pedidos + productos + usuarios)

```
WITH usuarios_unicos AS (  
  SELECT id, MIN(pais) AS pais  
  FROM usuarios  
  WHERE "$path" = (SELECT MAX("$path") FROM usuarios)  
  GROUP BY id  
)  
SELECT u.pais, p.categoria_nombre AS categoria,  
  COUNT(DISTINCT o._id) AS pedidos,  
  ROUND(SUM(CAST(i.cantidad AS DOUBLE) * CAST(i.precio_unitario AS DOUBLE)), 2) AS  
monto  
FROM v_pedidos_actuales o  
CROSS JOIN UNNEST(o.items) AS t(i)  
JOIN v_productos_actuales p  
  ON CAST(p.id AS BIGINT) = CAST(i.id_producto AS BIGINT)  
JOIN usuarios_unicos u  
  ON CAST(u.id AS BIGINT) = CAST(o.id_usuario AS BIGINT)  
GROUP BY u.pais, p.categoria_nombre  
ORDER BY monto DESC  
LIMIT 20;
```



SQL Ln 8, Col 45

Ejecutar de nuevo Explicar Cancelar Borrar Crear Volver a utilizar los

Resultados de la consulta Estado de la consulta

Completado Tiempo en cola: 107 ms Tiempo de ejecución: 1.816 seg Datos

Resultados (20)

Filtros de búsqueda

#	pais	categoria	pedidos	monto
1	Costa Rica	Juguetes	36	134979.58
2	Costa Rica	Deportes	40	134316.62
3	Guam	Deportes	26	133665.44
4	Estados Unidos de América	Juguetes	31	128592.55
5	Costa Rica	Alimentos	33	126547.73
6	Haiti	Alimentos	29	125881.42
7	Huelva	Alimentos	29	125655.38
8	Eslovaquia	Alimentos	32	124560.68
9	Noruega	Libros	38	123764.13
10	Francia	Libros	31	123262.77
11	Alicante	Juguetes	33	122045.82

Figura 25. Resultado de la Consulta 4 en Athena.

6. DESPLIEGUE EN AWS

El despliegue se realizó en AWS Academy Learner Lab siguiendo DEPLOYMENT.md. La arquitectura final separa producción, bases de datos e ingesta en cuatro instancias EC2 dentro de la misma VPC. Las dos MV de producción son réplicas idénticas: cada una ejecuta los cinco microservicios con docker compose y ambas usan la misma MV de bases de datos.

Instancia	Rol	Contenido	Acceso de red
cloudcommerce-backend	Producción 1 (réplica A)	ms-productos :8001, ms-usuarios :8002, ms-pedidos :8003, ms-checkout :8004, ms-analitica :8005	Puertos de aplicación solo desde el SG del balanceador
cloudcommerce-prod2	Producción 2 (réplica B)	Copia idéntica: ms-productos :8001, ms-usuarios :8002, ms-pedidos :8003, ms-checkout :8004, ms-analitica :8005	Puertos de aplicación solo desde el SG del balanceador
cloudcommerce-bd	Bases de datos (privada)	MySQL :3306, PostgreSQL :5432, MongoDB :27017	Sin IP pública ni 0.0.0.0/0; solo IP privadas de producción e ingesta
cloudcommerce-ingesta	Data Science	ingesta-productos, ingesta-usuarios, ingesta-pedidos	Rol LabInstanceProfile (S3)

Pasos principales

1. Lanzar las instancias EC2 (Ubuntu Server, t3.medium en backend y BD), instalar Docker y clonar el repositorio.
2. Levantar las bases de datos en cloudcommerce-bd y los cinco microservicios en cada una de las dos MV de producción con docker compose; cargar los datos ficticios una sola vez, porque las bases son compartidas.
3. Crear el ALB interno con un listener y un target group por microservicio (8001–8005), registrar en cada target group las dos MV de producción y verificar que ambos destinos estén Healthy.
4. Configurar ms-pedidos y ms-checkout con el DNS interno del ALB (no con IP fijas) para que el failover también cubra las llamadas entre microservicios.
5. Crear el VPC Link y las rutas de API Gateway con integración privada y CORS.
6. Publicar el frontend en Amplify con las URLs de API Gateway como variables de entorno.
7. Configurar la ingesta con las IP privadas de la MV de BD y ejecutar los tres jobs.

Instancias (4/9) Información									
Última actualización: Hace less than a minute Conectar Estado de la instancia Acciones Lanzar instancias									
Conjuntos de filtros guardados									
Elige conjunto de Filtr...									
Filtrar instancia por atributo o etiqueta (distingue mayúsculas y minúsculas)									
Name	ID de la instancia	Estado de la i...	Tipo de inst...	Comprobación de	Zona de dispon...	DNS de IPet pública	Dirección IP...	IP elástica	Direcciones I...
MV Desarrollo	i-067603f800e0eb0c	En ejecución	t3.micro	3/3 comprobador	us-east-1c	ec2-34-193-3-205.com...	34.193.3.205	34.193.3.205	disabled
cloudcommerce-bd	i-0098f30fc145a65bc	En ejecución	t3.medium	3/3 comprobador	us-east-1c	ec2-3-80-67-210.comp...	3.80.67.210	–	disabled
MV Pruebas	i-0bd5d9d43576d1cb1	En ejecución	t2.micro	2/2 comprobador	us-east-1b	ec2-3-87-0-111.comput...	3.87.0.111	–	disabled
cloudcommerce-backend	i-0142c0b53f7de2410	En ejecución	t3.medium	3/3 comprobador	us-east-1b	ec2-34-203-18-199.co...	34.203.18.199	34.203.18.199	disabled
MV Base de Datos	i-08f8fb89fa03ad35	En ejecución	t2.micro	2/2 comprobador	us-east-1b	ec2-54-83-143-5.comp...	54.83.143.5	–	disabled
MV-Prod 2	i-074db56d643ff4570	En ejecución	t3.micro	3/3 comprobador	us-east-1b	ec2-44-201-143-200.co...	44.201.143.200	–	disabled
MV Prod 1	i-051884c4399e329e7	En ejecución	t3.micro	3/3 comprobador	us-east-1a	ec2-100-60-80-251.co...	100.60.80.251	–	disabled
cloudcommerce-prod2	i-02e179efe1b736c20	En ejecución	t3.small	3/3 comprobador	us-east-1f	ec2-100-54-90-84.com...	100.54.90.84	–	disabled
cloudcommerce-ingesta	i-05655ec9519678a3f	En ejecución	t3.micro	3/3 comprobador	us-east-1c	ec2-3-93-31-85.comput...	3.93.31.85	–	disabled

Figura 26. Instancias EC2 del proyecto (marcadas) en AWS Academy.

Verificación de la redundancia

Cada target group tiene registradas las dos MV de producción, y ambas quedan en estado Healthy; además, las réplicas están en zonas de disponibilidad distintas (us-east-1b y us-east-1f), como muestra la Figura 27 para tg-pedidos. Con el sitio en uso, se detuvo una de las dos copias de ms-productos: el health check del ALB la marcó como Unhealthy y el tráfico pasó a la otra réplica, sin errores en el frontend.

tg-pedidos

Detalles

arn:aws:elasticloadbalancing:us-east-1:144548724267:targetgroup/tg-pedidos/472ce75e047a0058

Tipo de destino

Instancia

Tipo de dirección IP

IPv4

Protocolo : Puerto

HTTP: 8003

Versión del protocolo

HTTP1

Balancedador de carga

cloudcommerce-lb

2

Destinos totales

2

En buen estado

0

En mal estado

0

Sin utilizar

0

Anómalo

Distribución de destinos por zona de disponibilidad (AZ)

Seleccione los valores de esta tabla para ver los filtros correspondientes aplicados a la tabla Destinos registrados que aparece a continuación.

Destinos

Monitorización

Comprobaciones de estado

Atributos

Etiquetas

Destinos registrados (2)

Info

Los grupos de destinos enrutan las solicitudes a destinos individuales registrados mediante el protocolo y el número de puerto que especifique configuración de comprobación de estado del grupo de destinos. La detección de anomalías se aplica automáticamente a los grupos de destinos.

Filtrar destinos

☐	ID de instancia	Nombre	Puerto	Zona	Estado	Detalles
☐	i-02e179efe1b736c20	cloudcommerc...	8003	us-east-1f (use...	Healthy	-
☐	i-0142e0b53f7de2410	cloudcommerc...	8003	us-east-1b (us...	Healthy	-

Figura 27. Target group tg-pedidos con sus dos destinos Healthy: cloudcommerce-prod2 (us-east-1f) y cloudcommerce-backend (us-east-1b).

Grupos de destino (5/6)

Info

Novedades

Filtrar grupos de destino

☐	Nombre	ARN	Puerto	Protocolo	Tipo de destino	Balancedador de carga	ID de VPC
☐	TG-Prod80	arn:aws:elasticloadbalancing...	80	HTTP	Instancia	lb-prod	vpc-069c9f21ded4c1ebc
☒	tg-analitica	arn:aws:elasticloadbalancing...	8005	HTTP	Instancia	cloudcommerce-lb	vpc-069c9f21ded4c1ebc
☒	tg-checkout	arn:aws:elasticloadbalancing...	8004	HTTP	Instancia	cloudcommerce-lb	vpc-069c9f21ded4c1ebc
☒	tg-pedidos	arn:aws:elasticloadbalancing...	8003	HTTP	Instancia	cloudcommerce-lb	vpc-069c9f21ded4c1ebc
☒	tg-productos	arn:aws:elasticloadbalancing...	8001	HTTP	Instancia	cloudcommerce-lb	vpc-069c9f21ded4c1ebc
☒	tg-usuarios	arn:aws:elasticloadbalancing...	8002	HTTP	Instancia	cloudcommerce-lb	vpc-069c9f21ded4c1ebc

Figura 28. Target groups del ALB interno cloudcommerce-lb, uno por microservicio.

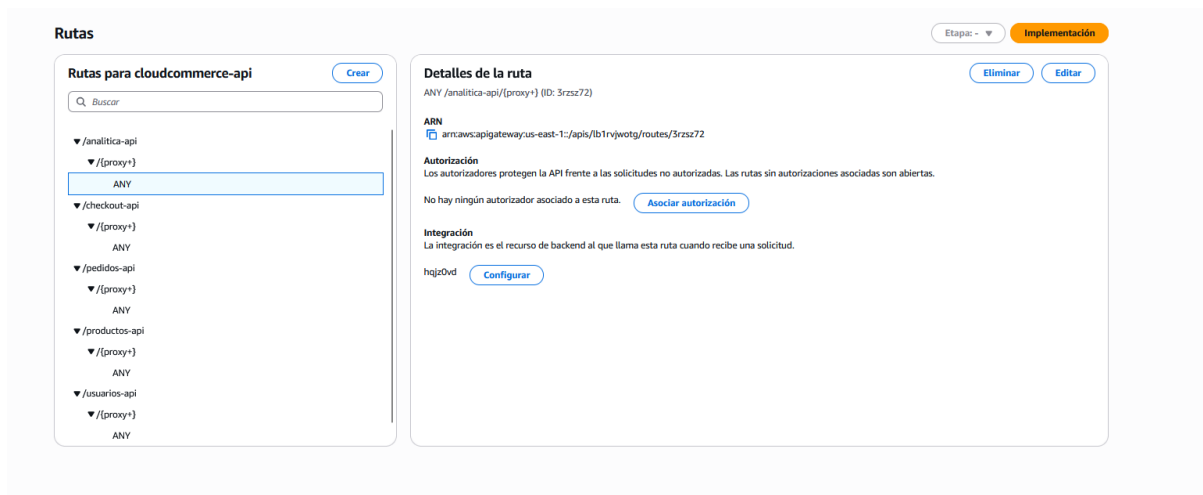


Figura 29. Rutas de API Gateway (cloudcommerce-api).

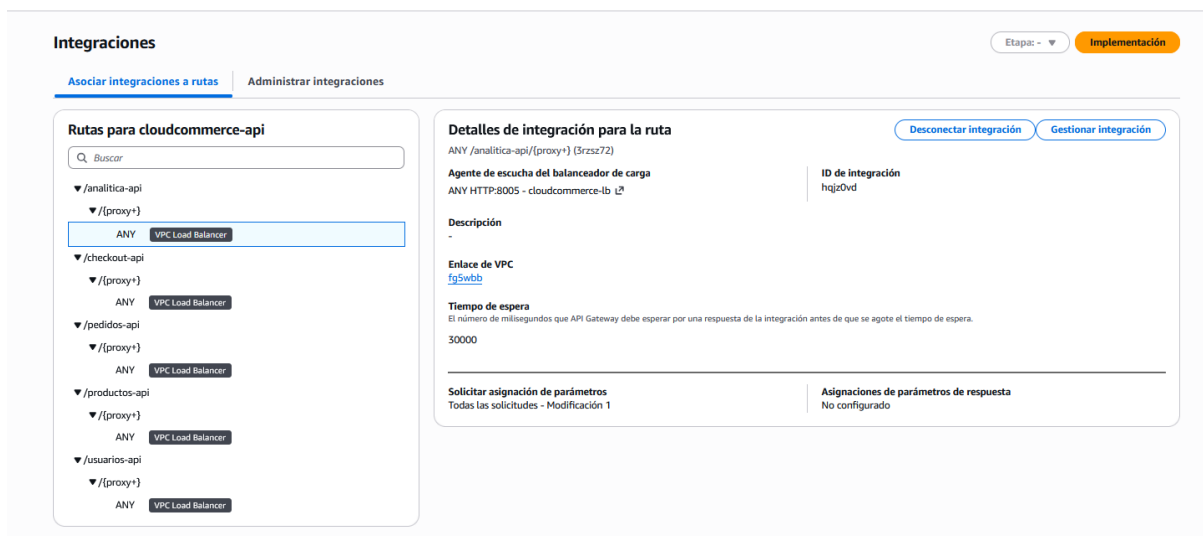


Figura 30. Integraciones privadas de API Gateway hacia el ALB a través del VPC Link.

7. LIMITACIONES Y TRABAJO FUTURO

- **Punto único de falla en los datos:** la redundancia cubre la capa de aplicación, pero las tres bases siguen en una sola MV (cloudcommerce-bd). Se propone migrar a servicios administrados con réplicas, como Amazon RDS Multi-AZ para MySQL y PostgreSQL y Amazon DocumentDB para MongoDB.
- **Seguridad de la aplicación:** las API no tienen autenticación y ms-usuarios no expone un login ni aplica hash a la contraseña recibida. Se propone incorporar autenticación con tokens y hash de contraseñas.
- **Ciclo del pedido:** el stock se valida en el checkout pero no se descuenta, y el estado del pedido no cambia después de pendiente. Se propone agregar actualización de stock y transiciones de estado.

- **Data lake:** cada ingesta genera un snapshot completo en el mismo prefijo; las vistas de Athena seleccionan el último. Una mejora es particionar por fecha para reducir lo que Athena escanea.

8. CONCLUSIONES

- La arquitectura en microservicios permitió combinar tres lenguajes y tres motores de base de datos, cada uno adecuado a su dominio.
- Aislar las bases en una MV privada y exponer todo mediante API Gateway, VPC Link y un ALB interno reduce la superficie de ataque.
- Replicar los cinco microservicios en dos MV detrás del ALB, y hacer que las llamadas internas también pasen por el balanceador, permite que el sistema siga respondiendo si cae una réplica; se comprobó deteniendo una copia de ms-productos con el sitio en uso.
- El pipeline S3 + Glue + Athena separa el análisis de la carga transaccional y habilita consultas que unen las tres fuentes.
- Abstractar el cliente de Athena (modo mock y real) permitió desarrollar y probar ms-analitica sin depender de credenciales de AWS.